Lecture 19

Trefoil v3: Scope

Group Work

```
(define (f x) (+ x 1))

(define (g n)
  (if (= n 0)
    0
    (+ n (g (- n 1)))))

(g 0)
(g 3)
```

```
| FnCall (f_name, arg_expr) -> (
      match lookup f_name env with
      | None -> failwith "unbound fn"
      | Some (VarEntry _) -> failwith "not a function"
      | Some (FnEntry (arg_name, body, def_env)) ->
          let arg_value = interpret env arg_expr in
          let extended_env =
            bind arg_name (VarEntry arg_value) def_env
          in interpret extended_env body)
```

```
| FnBinding (f_name, arg_name, body) ->
      bind f_name (FnEntry (arg_name, body, env)) env
```

1. Write the Bindings that correspond to the Trefoil program above.
2. What happens when the bindings are evaluated?

Group Work

```
(define (f x) (+ x 1))

(define (g n)
  (if (= n 0)
    0
    (+ n (g (- n 1)))))

(g 0)
(g 3)
```

```
FnBinding("f", "x",
  Add(Var "x", Val (Int 1)))

FnBinding("g", "n",
  If(Eq(Var "n", Val (Int 0)),
     Val (Int 0),
     Add (Var "n",
FnCall("g", Sub (Var "n", Val (Int 1))))
```

1. Write the Bindings that correspond to the Trefoil program above.

2. What happens when the bindings are evaluated?

Group Work

```
FnBinding("f", "x",
  Add(Var "x", Val (Int 1)))

FnBinding("g", "n",
  If(Eq(Var "n", Val (Int 0)),
     Val (Int 0),
     Add (Var "n",
FnCall("g", Sub (Var "n", Val
(Int 1))))


(g 0) FnCall("g", Val (Int 0))
(g 3) FnCall("g", Val (Int 3))
```

```
| FnCall (f_name, arg_expr) -> (
      match lookup f_name env with
      | None -> failwith "unbound fn"
      | Some (VarEntry _) -> failwith "not a function"
      | Some (FnEntry (arg_name, body, def_env)) ->
          let arg_value = interpret env arg_expr in
          let extended_env =
            bind arg_name (VarEntry arg_value) def_env
          in interpret extended_env body)
```

```
| FnBinding (f_name, arg_name, body) ->
      bind f_name (FnEntry (arg_name, body, env)) env
```

orrespond to the Trefoil program above.

2. What happens when the bindings are evaluated?

Group Work

```
| Some (FnEntry (arg_name, body, def_env)) ->
    let arg_value = interpret env arg_expr in
    let env1 =
    bind f_name (FnEntry (arg_name, body, def_env)) def_env
    in let env2 = bind arg_name (VarEntry arg_value) env1
    in interpret env2 body)
```

```
| Some (FnEntry (arg_name, body, def_env)) ->
    let arg_value = interpret env arg_expr in
    let env1 = bind arg_name (VarEntry arg_value) def_env in
    let env2 =
      bind f_name (FnEntry (arg_name, body, def_env)) env1
    in interpret env2 body)
```

Are these implementations the same? If not, write a Trefoil program that behaves differently depending on which implementation is used.

Group Work

```
(define (dsum l)
  (if
    (nil? l)
    0
    (if
      (cons? l)
      (+ (dsum (car l)) (dsum (cdr l)))
      l)))
```

What do each of the following calls evaluate to?

1. `(dsum 5)`
2. `(dsum (cons 1 (cons 2 (cons 3 nil))))`
3. `(dsum (cons 5 6))`
4. `(dsum (cons (cons 4 5) (cons 5 6)))`

Is it possible to write an OCaml function that does the same thing as `dsum`? Why/why not?